Neural Networks, Deep Learning and Applications
Report of FSD Tasks — Generative AI

Moritz Handrejk, Tim Rothe, Helmi Ghliss, Malak Mrini
TU Dresden - Summer Semester 2025

May 2025

Contents

Introduction

The reliability and safety of vehicle braking systems are critical components of automotive inspection protocols. As a result, braking systems must undergo thorough evaluation using sensor data that captures linear accelerations and angular velocities during dynamic test drives. This data serves as the foundation for detecting defects and ensuring compliance with safety regulations.

In this project, we explore the application of deep learning techniques to enhance the quality and utility of deceleration measurement data collected from vehicle braking tests. Specifically, we investigate two key areas: first, the use of Fully Convolutional Networks (FCNs), 2D Convolutional Neural Networks (2D CNNs), and Long Short-Term Memory (LSTM) networks for classifying braking system conditions. After that, we will focus on the application of Generative Adversarial Networks (GANs) and Recurrent Neural Networks (RNNs) to generate synthetic measurement data. The quality of this data will need to be evaluated and will be integrated to the training dataset to augment it.

Through dataset analysis, model training and evaluation, our work aims to improve model performance in classifying defective braking systems and to assess the effectiveness of our generative models by analyzing the quality of the synthetic data it produces and how much enriching our initial dataset with this new data is useful in enhancing training outcomes.

1 FCN, 2D CNN vs LSTM parameter tests and comparison

1.1 FCN

In this section, I implement a Fully Convolutional Neural Network (FCN) that will be used for training our model to recognize any issues with the braking system.

1.1.1 Chosen FCN model architecture

The chosen FCN processes architecture that has the following form: it takes the input of shape $(6, 128)$ (representing the 6 values of linear accelerations and angular velocities over the x,y,z axes for 128 series of measurements) and passes it through the following linear layers with ReLU activations.

Layer 1: Flatten Layer Converts the input tensor from shape $(6, 128)$ to a flat vector of size $768 = 6 \times 128$.

Layer 2: Linear Layer Fully connected layer: $128 \cdot 6 = 768 \to 128$

Layer 3: ReLU Activation Applies the ReLU function: $\text{ReLU}(x) = \max(0, x)$

Layer 4: Linear Layer Fully connected layer: $128 \to 64$

Layer 5: ReLU Activation Again, introduces non-linearity.

Layer 6: Output Linear Layer Fully connected layer: $64 \to 12$

The number of the hidden layers was chosen to be two as this seemed to show good results when it comes to the training accuracy.

1.1.2 The training data and the random split

The data used for the training was mainly extracted from the given file `train.pickle`. The data was then divided using a **random split** into *80% training data* and *20% testing data.* The random split was done 10 times giving us 10 randomly split training and validation data. This was done as preparation for the **Monte Carlo Cross Validation.**

The purpose of these split will become clearer later on. As we are trying to find the best combination of hyperparameters which give the best accuracy, having different training data resulting from this split makes sure the accuracy is **not dependent on the split** and rather on the effect of the **parameter combination.**

1.1.3 Hyperparameter combinations

The following hyperparameter combinations were used:

- **Optimizers**: SGD, Adam, RMSprop, Adagrad
- **Learning rates**: 0.001, 0.005, 0.01, 0.015
- **Epochs**: 7
- **Batch sizes**: 32, 64

The focus on **optimizers** and **learning rates** is justified by their *bigger influence on the accuracy.*

Other parameter combinations

Other combinations were tested but dropped to make visualization easier.

After trying different *epoch sizes,* setting the number of epochs to be **7** seemed to be meaningful for the training without reaching overfitting that often. To detect overfitting, the training accuracy and the evaluation accuracy were observed after each Epoch. In many cases **overfitting** could be detected as the *training* accuracy kept getting **higher** while the *evaluation* accuracy started getting **lower**.

Extremely **low learning rates** were also avoided due to different issues: A fast *plateau effect* is observed, making it look as if the model stabilizes too soon while in fact the learning is too slow. The number of epochs then needed for convergence will be significantly high. As a result, a lot of computation time will be wasted without meaningful results. The experiments for SGD and lr = 0.001 were also removed.

1.1.4 The training function

The training function takes as input:

- **Model**: which is the *FullyConnectedModel* in our case
- **Training dataloader**: which is made of 80% of the data taken from `train.pickle` using the random split
- **Test dataloader**: made of the remaining 20% of the data from `train.pickle`

- **Criterion**: or *loss function* which is the *CrossEntropyLoss*
- **Optimizer**
- **Number of epochs**
- **Experiment** (for the *comet* experiment)

The training function was designed to train the model, while keeping track of not only the training accuracy in each epoch, but also the *evaluation accuracy* using the 20% testing data.

1.1.5 The additional evaluation function

In addition to the evaluation done in each step parallel to the training - using the evaluation data generated using the random split - another evaluation function `evaluate` was created to check the actual test accuracy of the trained models in every split using the `Test.pkl` data.

The evaluation in this case is **independent of the training data,** which means the accuracy given is a good sign of how well is the training performing.

1.1.6 The experiment

After the decision on which model layers architecture will be used and fixing the number of epochs, thirty two major experiments were conducted. Each experiment uses the randomly split data and a combination of the hyperparameters. The accuracy was then observed over the 7 Epochs for the different splits.

The chosen hyperparameters proved to have major effects on the accuracy with accuracy going as high as **98%** for some combinations. This can be clearly observed in the following diagram:

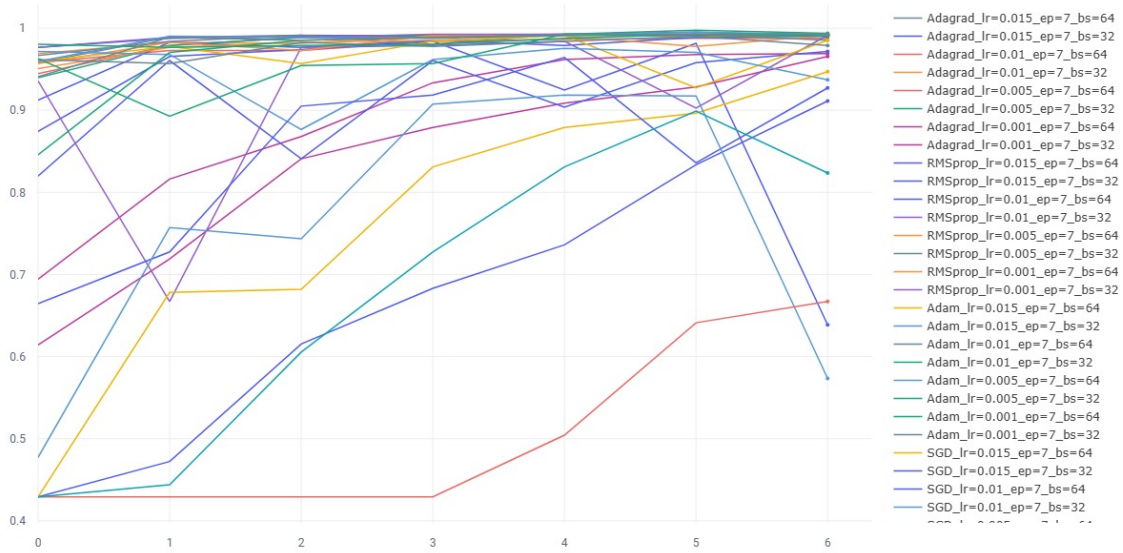


Figure 1: Evaluation accuracy per epoch for all models

In the next figures, the histograms with the *Evaluation Accuracy* using the *random split data* were plotted. Each one of them shows the effect of the **optimizer** choice, the **learning rate** on the accuracy in average.

The *Adam* optimizer proved to be the best with an average accuracy higher than *97%*. The *SGD* gave lower values with an average accuracy slightly higher than **70%**. The *batch size 32* gave better accuracy in average and the *learning rate 0.005* was associated with the best results.

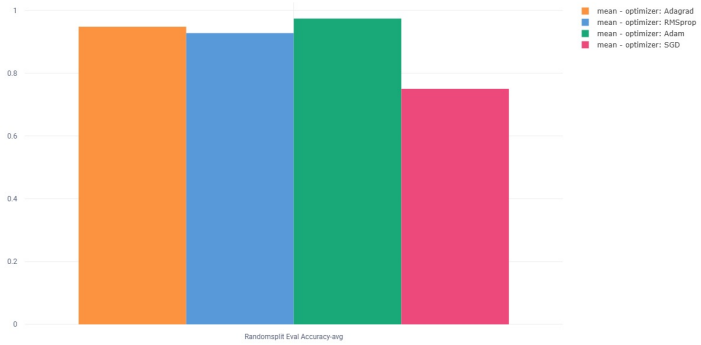


Figure 2: Random split evaluation accuracy, for each optimizer

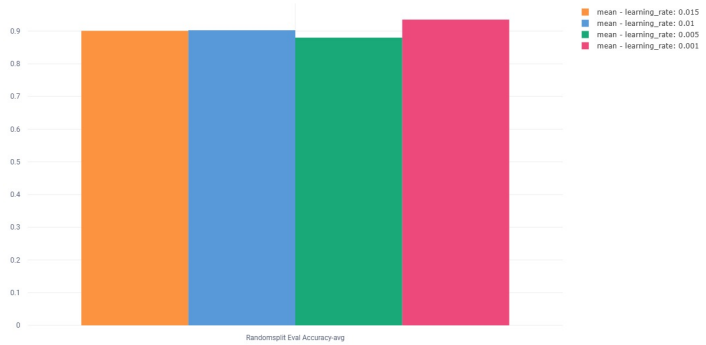


Figure 3: Random split evaluation Accuracy, for each learning rate

The random split, however, **preserves the distribution of the data** (because of the Monte Carlo principal). This makes the accuracy higher than it would be for an independent data set.

1.1.7 Using the test dataset for evaluation

After each training, the `test.pickle` dataset was used to observe how well does the model performs on different data. The **highest** average accuracy in this case was equal to **47%** and was attained for the following hyperparameter combination : *SGD, lr=0.015 , batch size = 64.* The **lowest** values were observed for *Adam*, reaching the minimal average value of **33,7%** for *lr=0.01* and *batch size = 64.* This shows a clear difference to our results when using the random split.

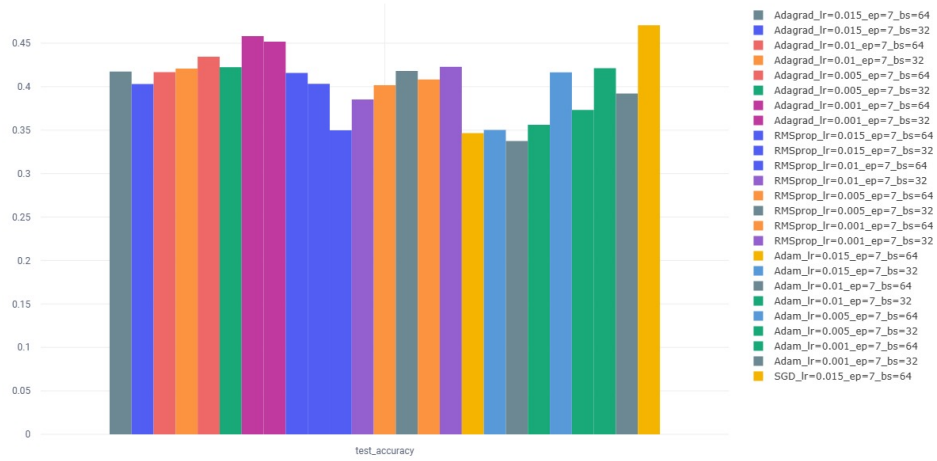


Figure 4: Test data evaluation accuracy average over the 10 splits

Next we plot the average (over the splits) of the evaluation Accuracy mean (over the experiments with the common parameters) for each model to see which parameters had better performance in average.

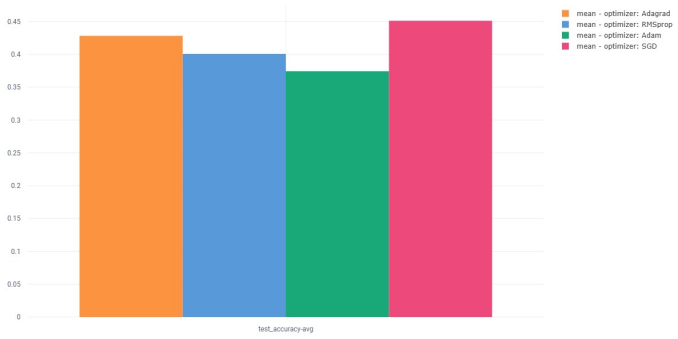


Figure 5: Test data evaluation Accuracy, for each optimizer

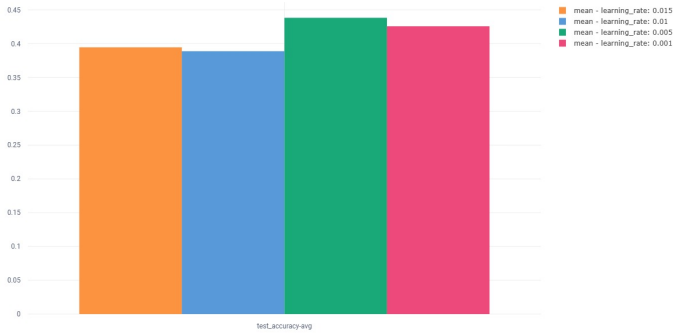


Figure 6: Test data evaluation Accuracy, for each learning rate

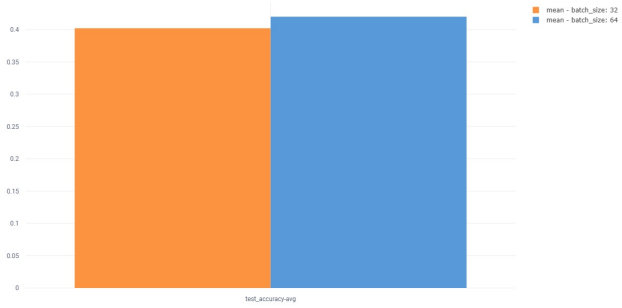


Figure 7: Test data evaluation Accuracy, for each batch size

The Histograms here show a difference to the ones using the random split and give a different choice of parameters for getting the optimal accuracy.

Given the difference between the test data set and the training data set, the random split evaluation can be a better indicator of how well does the model performs for similar data. However, for relatively different data , more **stable** models are a better alternative, with the *SGD* optimizer being the best choice with it is accuracy between 40% and 51% (for the different splits).

1.2 CNN

In this part, we discuss the implementation of a 2D CNN that can classify the condition of brakes, involving 2D input data of shape (128,6) that is reshaped as (1,128,6) to match the expected input format for convolutional layers. The model outputs predicitons across the 12 possible classes.

1.2.1 Architecture

- The first 2D convolutional layer applies 32 filters of size 3x3 with padding in order to preserve spatial dimensions. A ReLU activation function is applied after, as well as 2x2 max pooling, halving the shape to (32,64,3).
- The same process is again applied in the second 2D convolutional layer, this time with 64 filters, resulting in a feature map of shape (64,32,1) after a second pooling.
- The output of the final pooling layer is flattened into a 1D vector of size 2048 to prepare for the fully connected layers.
- The first fully connected layer is a dense layer with 128 neurons and ReLU activation. Finally, the model outputs class scores through a fully connected layer with 12 output neurons for classification.

1.2.2 Training the model

At first, I manually tried a few simples combinations of some hyperparameters, namely *the optimizer* and the *learning rate*, to investigate how the model performed before diving deeper into a bigger comparison experiment.

In my case, the CNN classifier converged rapidly (around 99% accuracy in 5 epochs), with a steady decline in training loss. This may indicate that the model is effectively learning. However, these results must be evaluated on unseen test data to ensure generalization and absence of overfitting.

1.2.3 Evaluating the model

After the training, the model is evaluated on the whole `test.pkl` file. I noticed that despite the very high training accuracy, the model's test accuracy was very low. This indicates a significant generalization problem : the model is probably overfitting to dominant patterns in the training data and also struggling with the class imbalance the dataset shows.

```
Test Accuracy: 0.1798

Classification Report:
              precision    recall  f1-score   support

           0     0.6866    0.1551    0.2531       593
           1     0.7059    0.1154    0.1983       104
           2     0.9412    0.3048    0.4604       105
           3     0.0000    0.0000    0.0000         0
           4     0.1375    0.4272    0.2080       103
           5     0.0000    0.0000    0.0000        96
           6     0.0000    0.0000    0.0000         0
           7     0.0000    0.0000    0.0000         0
           8     0.0000    0.0000    0.0000         0
           9     0.0000    0.0000    0.0000         0
          10     0.0000    0.0000    0.0000         0

    accuracy                         0.1798      1001
   macro avg     0.2246    0.0911    0.1018      1001
weighted avg     0.5929    0.1798    0.2402      1001
```

Figure 8: Example of classification report

In this example, the classification report in Figure 8 shows that the model performs poorly overall, with a test accuracy of only 17.98%. It also shows that only a few classes were predicted with any reliability (class 2 for example), but very low recall across most, indicating many true instances are missed, while most others were either completely unrecognized or frequently misclassified.

The confusion matrix in Figure 9 reveals that predictions are **heavily biased** toward a few dominant classes. Classes with highest number of samples - *such as class 0* - are often misclassified, reflecting confusion between feature patterns, while minority classes are either completely unrecognized or confused with dominant classes, which shows that the model fails to distinguish or even detect them.

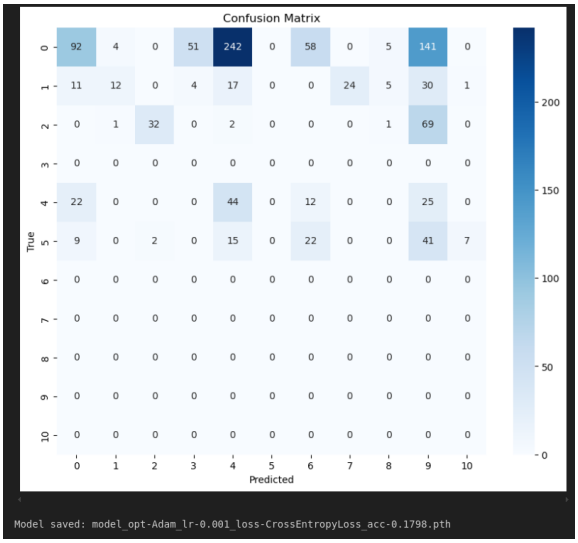


Figure 9: Example of confusion matrix of model predictions on the test set

In order to address these issues, many potential methods can be explored, such as *regularization techniques* (e.g dropout), *random split, early stopping,* and *finetuning of hyperparameters.*

1.2.4 Improvements

To improve the performance of my CNN model, I implemented a structured hyperparameter search process. This big experiment consisted in testing a range of combinations for key model hyperparameters, including *learning rate, batch size, dropout rate,* and *optimizer type* listed in Table 1.

Hyperparameter	**Values**
Dropout rate	0.0, 0.3, 0.5
Batch size	16, 32, 64
Optimizer	Adam, SGD, AdamW
Learning rate	0.0001, 0.0005, 0.001, 0.005

Table 1: Explored hyperparameters

However, I kept using the same loss function which is *Cross Entropy Loss* as it is an interesting and widely used loss function for multi-class classification. Since

the training set showed great class imbalance, I computed **class weights** and integrated them into the loss function. This helped ensure that underrepresented classes received proportional gradient attention.

Using a random split on the `train.pkl` dataset *(80% for training, 20% for evaluation)*, I evaluated over 100 configurations. This extensive process allowed me to identify which hyperparameter combinations yielded higher generalization performance.

Alongside with the random split, I also implemented **early stopping** in the training loop, which halts training if the validation loss does not improve for a specified number of consecutive epochs, preventing overfitting. Loss and accuracy metrics are recorded during the training for both training (80%) and validation (20%) datasets to track progress. By this, we make sure that the model's generalization ability is assessed.

This process enhances the importance of the random split, as this parallel evaluation is necessarily done with the data from the training file, not leaving any room for *data leakage*. The final evaluation of each trained model was performed using the untouched `test.pkl` dataset. This ensured that accuracy results reflect the true generalization capability of the models and helped detect any overfitting observed during training.

Accuracy scores for all configurations were logged and compared. I also logged *training times* to evaluate computational cost. Results can be visualized using parallel coordinate plots as in Figure 25, which made it easier to identify patterns in how hyperparameters impacted performance.

In general, the *Adam* optimizer consistently yielded better performance than the other optimizers. Its adaptive learning rate mechanism likely helped stabilize training. As for learning rates, *0.001* is commonly effective. Smaller values tended to *underfit*, while larger ones caused *erratic behavior*. Among all tested batch sizes, the largest batch size gave the best results. Finally, for dropout, *0.3* seemed to be the perfect percentage : it's high enough to prevent overfitting, but not too aggressive to hinder learning.

To summarize, the best-performing model achieved a test accuracy of **50.25%** using the **Adam** *optimizer*, a *learning rate* of **0.001,** *batch size* of **64,** and *dropout rate* of **0.3**. Considering training time, which varied across configurations from approximately 10 to 35 seconds, the best configuration gave a model that trained in around **25 seconds,** placing it in the middle range of training durations.

While the initially selected CNN configuration achieved a peak test accuracy of 50.25% on a single split, I thought a more rigorous evaluation across **10 different random dataset splits,** each with **10 repeated training runs,** is necessary to evaluate how representative that result we achieved was. Indeed, this experimentation yielded a lower mean test accuracy of around **40%** as deducted from Figure 10.

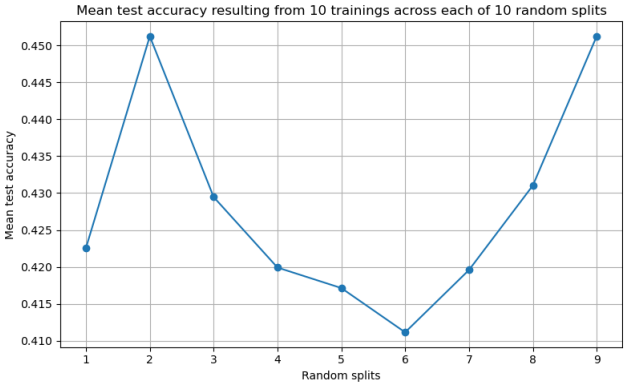


Figure 10: Mean test accuracy of each 10 trainings over each of the 10 random splits

This reduction suggests that some data splits present **greater learning challenges**. This performance drop across different data splits is expected, as CNNs are sensitive to variations in data distribution, class balance, and sample complexity. Some splits may contain **harder-to-learn patterns** or less representative training data, leading to lower generalization. This variability highlights **the importance of evaluating models across multiple random splits** to obtain a **reliable estimate of true generalization performance,** as emphasized in standard deep learning methodology (see I. Goodfellow, Bengio, and Courville 2016, , Chap. 5 and 7).

Within this variability, I wanted to find the best achievable performance of the model. For that, several runs of the experiment explained just above were done. I could obtain a decent model that achieved a test accuracy of **49.75%**, completing training in **23.53 seconds**, which is consistent with the expected runtime range discussed earlier.

In particular, the model now achieves strong performance on classes 0, 1, and 2 — all of which are well-represented in both training and test sets — with relatively high F1-scores and recall.

```
Training completed in 23.53 seconds

Test Accuracy: 0.4975

Classification Report:
              precision    recall  f1-score   support

           0     0.7807    0.6425    0.7049       593
           1     0.8704    0.4519    0.5949       104
           2     0.8654    0.4286    0.5732       105
           3     0.0000    0.0000    0.0000         0
           4     0.4194    0.1262    0.1940       103
           5     1.0000    0.1250    0.2222        96
           6     0.0000    0.0000    0.0000         0
           8     0.0000    0.0000    0.0000         0
           9     0.0000    0.0000    0.0000         0

    accuracy                         0.4975      1001
   macro avg     0.4373    0.1971    0.2544      1001
weighted avg     0.7828    0.4975    0.5808      1001
```

Figure 11: Classification report of best reproduced model (49.75% accuracy)

However, performance drops sharply for classes with lower support in the test set. For instance, class 5, despite a perfect precision of 1.0, has a recall of only 12.5%, indicating that predictions for it are rare and possibly incidental. More critically, some classes present in the training set are **completely absent** from the **test** set, which limits the model's opportunity to demonstrate generalization on them.

The confusion matrix confirms that most errors involve **misclassification** into **majority** classes, particularly 0 and 9. Overall, this result confirms that meaningful improvements were achieved, but also highlights how dataset imbalance create fundamental limitations on model evaluation and learning.

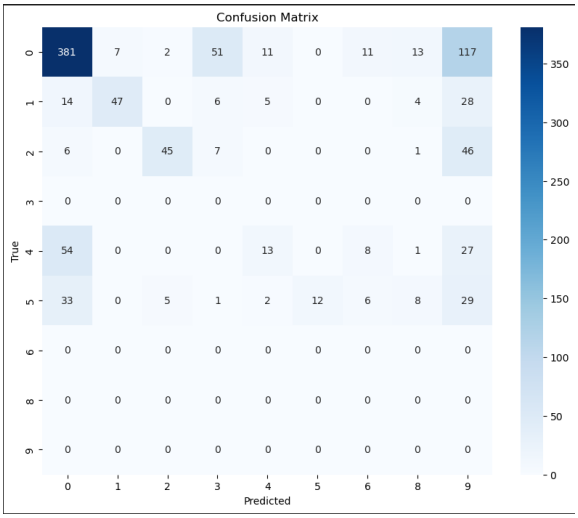


Figure 12: Confusion matrix of best reproduced model (49.75% accuracy)

1.3 LSTM

1.3.1 Methodology and Hyperparameters

The LSTM (Long Short Term Memory) model takes a 6-dimensional input (3 for directional velocity and 3 for rotational velocity) and feeds it into an LSTM layer. The *number of hidden layers*, their *dimension*, the *dropout rate* between layers, and the *batch size* are all **variable hyperparameters.** The output from the final hidden state is passed through a fully connected linear layer, which maps to 12 output values corresponding to the possible states of the brake system.

I use the **Adam optimizer**, which is commonly regarded as a strong choice for LSTMs. This introduces two additional hyperparameters: the *learning rate* and *weight decay*. For the loss function, I use **cross-entropy loss**, which is well-suited for multi-class classification tasks as it directly quantifies the difference between predicted probabilities and true class labels. The objective is to find the hyperparameter configuration that maximizes classification accuracy.

To start, I created a list of sensible ranges for each hyperparameter:

Hyperparameter	Values Considered
Hidden dimension	16, 32, 64, 128
Number of layers	1, 2, 3
Dropout rate	0.0, 0.1, 0.3, 0.5
Batch size	16, 32, 64
Learning rate	1e-4, 1e-3, 1e-2
Weight decay	0.0, 1e-5, 1e-4

Table 2: Sensible hyperparameters to explore

To ensure reliable model comparisons, I aim to use *random splitting*, where each model is trained and validated across 10 different random splits. Importantly, these 10 random splits remain fixed throughout all experiments to maintain consistency and enable fair comparisons between models. Additionally, I implement *early stopping* in each split to prevent overfitting by halting training when validation accuracy no longer improves.

1.3.2 Initial Random Search

Since it isn't computationally feasible to evaluate all possible hyperparameter combinations—especially when each model is trained 10 times with early stopping—I began with an initial experiment : it consisted of 100 training runs using randomly selected hyperparameter combinations from Table 2. These runs did not include random splitting or early stopping. Each model was trained for a fixed number of 10 epochs.

The results can be seen in Figure 13. The highest test accuracies reached around **60%**, but results varied significantly across runs.

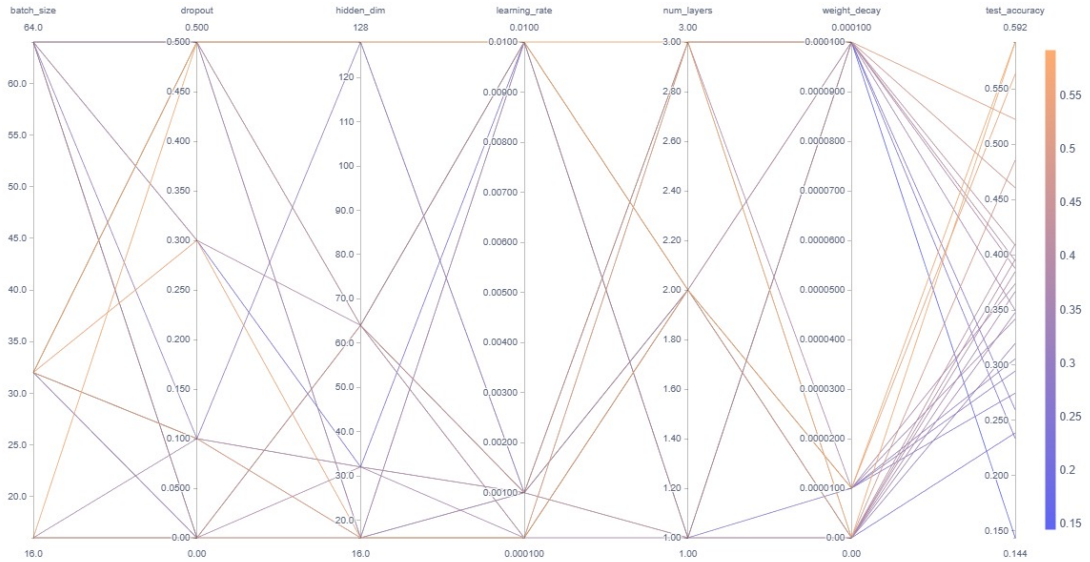


Figure 13: All 100 randomly selected training runs without random splitting or early stopping from high test accuracy to low test accuracy

By looking at each hyperparameter seperaterly, it becomes clear that a *batch size* of **64** and a *hidden layer count* of **1** consistently *underperformed*. These will not be considered in future models.

As for the remaining hyperparameters, the limited amount of data doesn't provide clear guidance. With more runs, it's likely that further exclusions could be justified. However, due to computational constraints, that's not feasible here.

Likewise, it's not practical to do a full grid or random search under the more rigorous setup involving random splitting and early stopping. Instead, I resort to comparing individual models under those conditions.

1.3.3 Targeted Experiments

In this section, I take the best-performing configuration from the initial random search and evaluate performance using random splitting and early stopping. The configuration is shown in Table 3:

Hyperparameter	Value
Input dimension	6
Number of classes	12
Hidden dimension	128
Number of layers	2
Dropout	0.5
Batch size	32
Learning rate	0.01
Weight decay	0
Max epochs	20
Patience	3

Table 3: Best-performing configuration from initial random search

Running this configuration on 10 random splits yielded a test accuracy close to **60%** in four out of ten runs (Figure 14). As seen in Figure 15, some runs showed steady improvements in evaluation accuracy over epochs, while others **plateaued** early at **44.39%**.

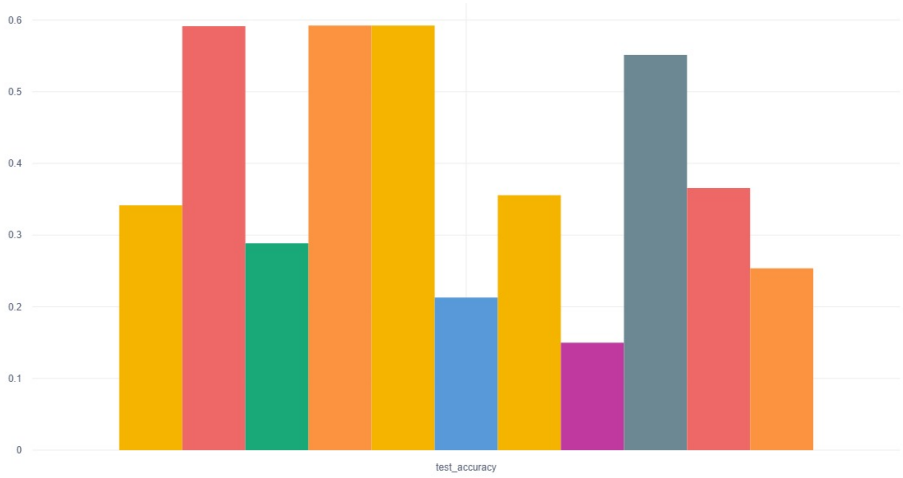


Figure 14: Test accuracy across 10 different random splits

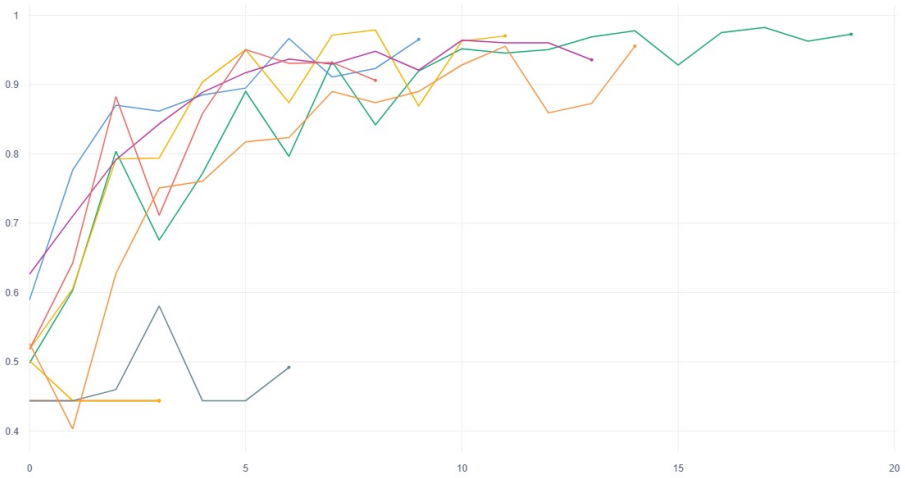


Figure 15: Validation accuracy across 10 random splits over 20 epochs

A closer look shows that in these *plateauing* cases, the model gets stuck in a local minimum — always predicting class **0**, which means all brakes are intact. This is the most common state in both the training and test sets. Since about **60%** of the test set consists of samples with all brakes intact, this strategy leads to deceptively good test accuracy. However, always guessing 0 is clearly not the goal. After trying many other unsuccessful adjustments, I introduced *class weights* into the *cross-entropy loss function,* which successfully prevented the model from always predicting zero. This approach assigns higher penalties to errors on underrepresented classes, encouraging the model to learn more balanced predictions.

It is worth mentioning that here I changed the patience to 5 and the maximum number of epochs to 50. However, no model ever trained for more than 20 epochs in practice.

After these changes and further tuning of each hyperparameter, I arrived at the following final configuration:

Hyperparameter	Value
Input dimension	6
Number of classes	12
Hidden dimension	128
Number of layers	2
Dropout	0
Batch size	32
Learning rate	0.0001
Weight decay	0.0001
Max epochs	50
Patience	5

Table 4: Final configuration with class-weighted loss

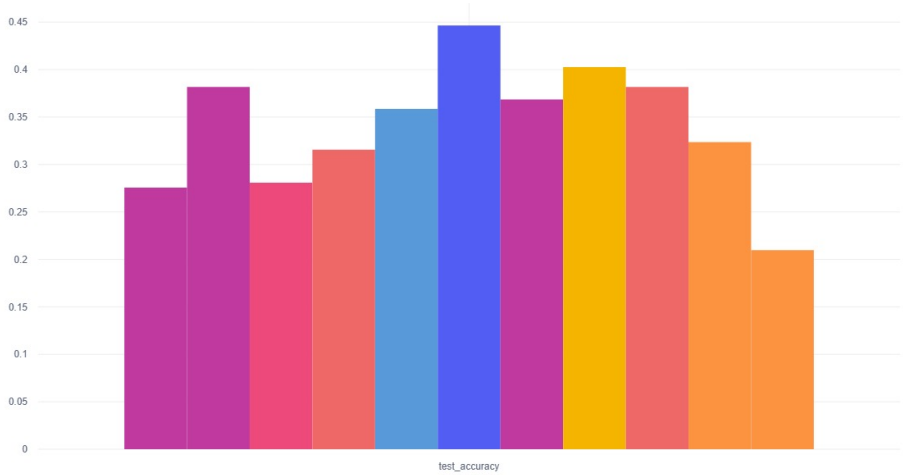


Figure 16: Test accuracies of the best performing LSTM model on 10 random splits

This led to an accuracy of only **34,69%**. So while the model no longer defaults to guessing 0, its overall performance remains **underwhelming**. In fact, just guessing class 0 still leads to better results than the trained model, highlighting the limitations of the dataset more than anything else when using LSTMs.

1.4 Comparisons and conclusion

In this previous part, three deep learning architectures were evaluated for the task of classifying vehicle braking system conditions.

The **FCN** demonstrated strong performance on training-like data with a random split, achieving over 95% accuracy in some configurations, but its performance dropped significantly on unseen test data, peaking at 47%, indicating overfitting and limited generalization. The **LSTM** initially showed promising results with some runs approaching 60% accuracy; however, it frequently defaulted to predicting the dominant class due to class imbalance, and even after adjustments like *class-weighted loss,* its average test accuracy remained low at 34.69%. The **2D CNN** achieved the best balance between accuracy and consistency. It reached a peak test accuracy of 50.25% and maintained a more stable performance across random splits (average around 40%), especially when tuned with dropout and the Adam optimizer.

Given its better generalization capability, more moderate variance, and overall best test accuracy, the **2D CNN** is selected as the preferred architecture for the next phase of the project. The upcoming focus will be on improving its performance by enriching the dataset using synthetic data generated through generative models.

2 Generative AI

In this section we investigate the **generation** of time series data using Deep Learning algorithms. This can be useful to **improve** our classification networks from the previous section by extending our training dataset with synthetic data.

2.1 Quality Estimation

In order to investigate the quality of our generated data we need to devise a way to estimate the quality. For that, we can compute the *arithmetic mean* and *standard deviation* of every measurement at every time step. The resulting graphs for the entire training dataset are shown in Figure 17.

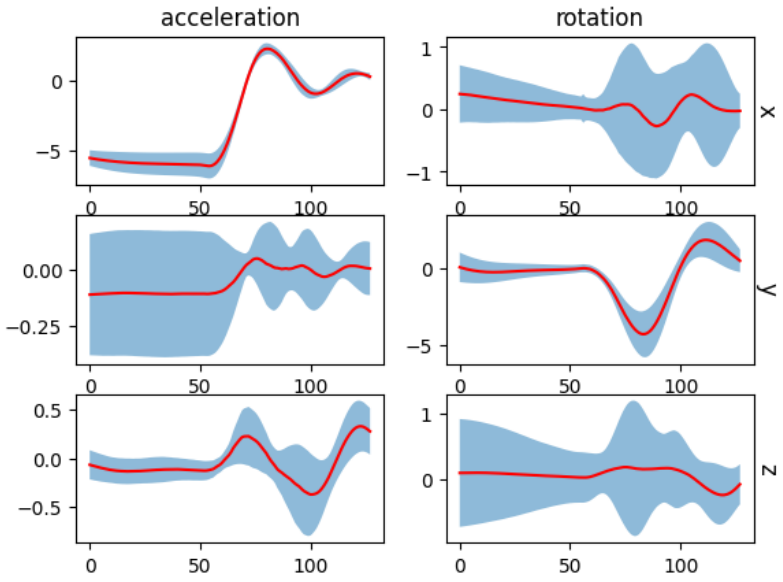


Figure 17: The deviation plot over time. The red graph is the mean μ_t over time t. The blue area is the space between $\mu_t - \sigma_t$ and $\mu_t + \sigma_t$.

From this figure we can analyze the general behavior (*distribution*) of every feature in the time dimension. We can see that the graph of every feature remains constant for the first 50 time steps, distinguishing the constant deceleration during braking. Some distinct curve behavior can be observed after the braking was performed. For example, the *acceleration* in *x-direction* and *rotation* in *y-direction* have **distinct curves after braking.**

The deceleration in x-direction remains constant at around -5 for the first 50 time steps. After around 50 time steps the acceleration quickly rises to a peak around 1 after 75 time steps. Following the initial acceleration the car decelerates again to just below 0 and recovers more slowly to no acceleration. Rotation in y-direction remains at 0 for the first 50 time steps. After the initial acceleration impact it quickly decreases to an average minimum around -5 before quickly increasing to a maximum value just above 0. After this impact, the rotation recovers slowly to the initial value. Later we can show that these **distinct features** of the curves are *learned quickly* by the generator network in a few epochs. Other more intricate features of the graphs take longer to learn.

Taking the **mean** curve of the entire training dataset is **heavily biased** against the *most common* braking classification. To measure the **quality** of our time series with a given braking classification, we can use the mean curve of all real data which have the given braking classification. Putting this all together, a measure for the quality of our artificial data given a braking classification can be given by the **least squares distance of the artificial data from the mean curve of its braking classification.**

Given a label $s \in \{0, \dots, 11\}$ and mean time series $\mu_{t,s} \in \mathbb{R}^6$ for $t \in \{0, \dots, 127\}$, we can now define r as

$$r((a_t), s) = \sum_{i=1}^{6} \frac{1}{T} \sum_{t \in T} (a_t^i - \mu_{t,s}^i)^2.$$

a measure for the quality of an artificial time series labeled s.

This measure does not capture the *standard deviation* from the original data and *large temporal correlations* within the labeled data itself. For our purposes, this measure proved good enough to estimate how good our synthetic data is. In addition, the mean curve also gave us an idea how to *judge* the artificiality during the training process with our own eyes. It is also possible to improve this function by taking the *distribution of the generator* into account by computing the mean curve of the whole generated dataset (latent space) and comparing both curves. Extending this approach to account for *covariance* leads to a general measure called **Frechet inception distance** (introduced by Heusel et al. 2017), which is widely used as a measure of quality in many generative applications now.

2.2 Using Generative Adversarial Networks

A common method to create synthetic data is by using a Generative Adversarial Network (GAN). Initially introduced by I. J. Goodfellow et al. 2014, it is an architecture specifically designed for generating artificial output. A **GAN** consists of two deep neural networks: a *generator* network and a *discriminator* network.

The *generator* network **generates artificial data** from random noise, from the so-called *latent space*. The *discriminator* network **evaluates the output of the generator** network on the *artificiality* of the data. The aim is to improve the generator network, such that it can be used to generate realistic time-series from random noise.

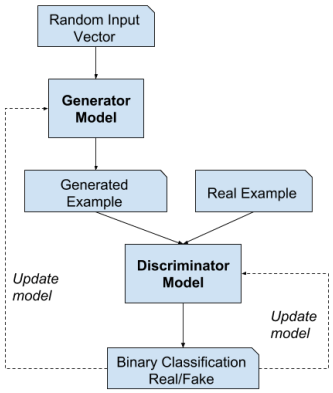


Figure 18: Architecture of a Generative Adversarial Network

Although generating artificial data might be enough for some use cases, it is not enough for generating data with specific classifications (like our braking condition). Therefore, we must use a so-called **conditional GAN.** In this architecture the generator and discriminator is supplied with additional information (the '*condition*'), which is then used to generate artificial data. We applied this architecture to generate **classification-specific data** from the same neural network.

Models

As a general architecture, we chose a *1D-CNN*-based approach for designing the generator and discriminator network. In order to distinguish the 12 braking classifications, an *embedding vector* is used with variable dimension.

- **Generator Network**. The generator network is *1D-deconvolution* based network with *batch-normalization*. The latent vector has a dimension of 100, where each sample is gaussian distributed with variance 0.02. The reason for this smaller variance is to **reduce the discontinuities** of the generated data.
  - **Embedding**: An embedding vector with dimension 16 is chosen. The embedded vector is concatenated with the latent vector and projected into the right initial dimension.
  - **Projection**: `Linear` $\rightarrow$ `ReLU`, where the final output vector has dimension $256 * 8$, which is reshaped to $(256, 8)$.
  - **CNN Decoding**: `ConvTranspose1d` $\rightarrow$ `BatchNorm1d` $\rightarrow$ `ReLU` applied 4 times sequentially. The dimensions are $(256, 8) \rightarrow (128, 16) \rightarrow (64, 32) \rightarrow (32, 64) \rightarrow (6, 128)$. Finally, `ConvTranspose1d` at every decoding step has kernel size 4, stride 2 and padding 1.
- **Discriminator Network**. The discriminator is again *convolution-based* and downscales similarly to the generator function. The output of the discriminator is a value between 0 and 1, where 1 classifies the input as **real** input and 0 as **artificial** input.
  - **Embedding**: An embedding vector with dimension 6 is chosen.
  - **Projection**: The embedding vector is concatenated at every time step of the time series.
  - **CNN Encoding**: `Conv1d` $\rightarrow$ `BatchNorm1d` $\rightarrow$ `LeakyReLU` is applied 4 times sequentially. For the first layer of decoding `BatchNormalization` is omitted and the last layer omits the activation function. Every `ConvTranspose1d` instance has kernel size 4, stride 2 and padding 1. Every `LeakyReLU` instance has a negative slope of 0.2. The last convolution reduces the feature space to a single value where a sigmoid function is applied.

Training process

Practically, the training process can be described as a (mathematical) **game**, where each player (the *discriminator* and *generator*) tries to **minimize their loss.** We used the *binary cross-entropy loss* function in our training:

$$L_D(y_{\text{real}}, y_{\text{fake}}, c) = -\mathbb{E}[\log(y_{\text{real}}(c))] - \mathbb{E}[\log(1 - y_{\text{fake}}(c)))]$$
$$L_G(y_{\text{fake}}, c) = -\mathbb{E}[\log(y_{\text{fake}}(c))]$$

where

$$
\begin{aligned}
y_{\text{real}}(c) &= D(x_{\text{real}}, c) \\
y_{\text{fake}}(c) &= D(G(x_{\text{latent}}, c), c) \\
c &\in \{0, \ldots, 11\} \text{ braking classification}
\end{aligned}
$$

Together, the training solves a **two-player minimax game.** Due to the game-theoretic nature of the training process, various events and steady states can occur. If the *discriminator* is too **strong**, the discriminator *loss* is very **low**, which means that the **gradient** of the discriminator function *vanishes* and no training improvements can be seen. On the other hand, if the *generator* is too **strong**, the training process is hampered by the **slow progress** of the *discriminator*. In addition to striking a balance of power between the discriminator and generator, steady states can occur which reduce the perceived quality of the synthetic data.

The following configuration was used:

- The `Adam` optimizer with learning rate 10^{-4} for both generator and optimizer was used. Multiple learning rates and also differing learning rates were tried, but 10^{-4} resulted in the most consistent loss behavior.
- Although initial results can be seen after a few epochs, the training ran for 300 epochs. Every few epochs a sequence from a specific braking classification is generated and inspected.
- To strengthen the learning of the discriminator network, 3 discriminator optimization steps for each generator optimization step were computed.

The entire training took around **20 minutes** with **GPU** acceleration.

Evaluation

The *training loss-plot* can be seen in figure 19. From this figure we can see that the discriminator loss is consistently **higher** than the generator loss. Although there are outliers and large jumps in the plot, the loss plot remains *somewhat stable.* It is also possible to inspect the improving quality of the generated output visually. A sample evolution can be found in section 3.

Label	Avg. Error
0	0.4561
1	3.8970
2	1.6376
3	2.4446
4	0.4103
5	0.4668
6	0.4429
7	1.7961
8	5.9306
9	2.1232
10	4.5096
11	10.3525

Table 5: Average generation error per label

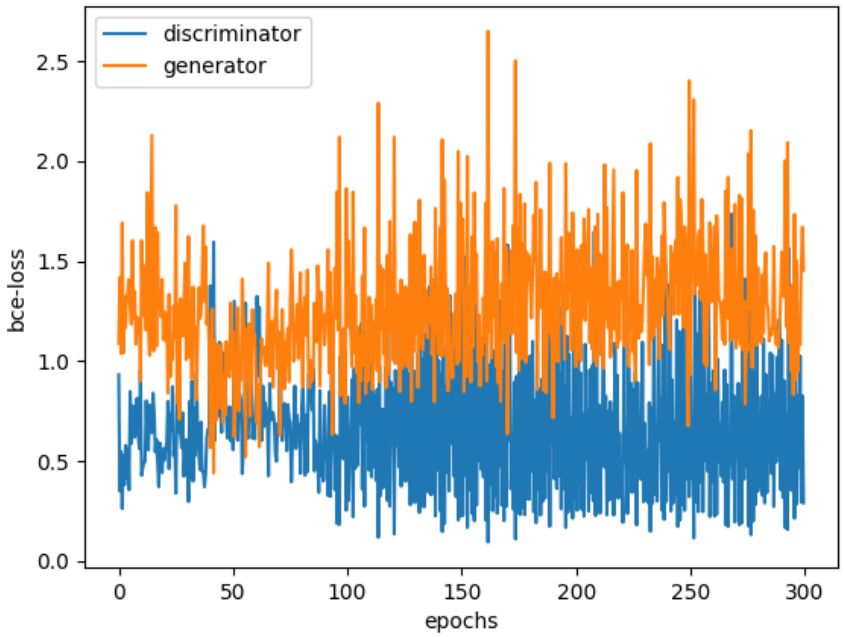


Figure 19: Loss plot after 300 epochs of training

In addition the average error as explained in section 2.1 was computed for $n = 1000$ samples per label. The result can be seen in table 5. We can see that the generator performs notably **well** for certain labels *(0, 4, 5 and 6)* and for other labels *(1, 8 and 11)* particularly **bad**. This may be due to a lack of training on these labels as even in the training data there is only a small sample size given. In addition, this may be due to our measure, which may not capture temporal correlations and large deviations in the mean curve itself. Visually inspecting the output, the CNN generator also does not generate entirely smooth sequences as opposed to the real data. This is due to its architecture, which is based on (rather small) deconvolution steps. It may take further training to minimize these errors.

We can take additional post-processing steps, which would increase the smoothness of the data. Firstly, one could introduce a post-processing step which increases the smoothness by *gaussian smoothing.* It is also possible to increase the temporal resolution of the synthetic output and apply a downsampling algorithm as a post-processing step. This comes at greater computational cost.

Both of these methods should result in an overall smoother time series, which may increase the usefulness of the synthetic output, but due to lack of time, these were not further investigated.

2.3 Using RNN

For generating sequences using a RNN, I decided on an **LSTM** with the 6 sensors as inputs. Using an LSTM here makes sense because time series data is inherently **sequential**, and LSTMs are well-suited to capturing dependencies over time.

The output of the last hidden layer of the LSTM is passed through a fully connected linear layer to produce **six outputs** — which represent the predictions for the next timestep. For all following models there are 2 hidden layers each of size 128 and the training is performed over 10 epochs.

While during training the LSTM processes the entire input sequence at once, it internally maintains recurrent connections across timesteps. This means each output is still influenced by previous states, maintaining *temporal continuity.*

I used **MSELoss** as the loss function because it is well-suited for comparing continuous sequences, such as sensor values. Unlike *classification* problems — where

losses like *cross-entropy* are more appropriate — MSE directly **penalizes deviations** in predicted versus actual values. Also, I again use the **Adam optimizer** as I have done in Section 2.3.

After the training of the model is finished, I give the model *random start values* — normally distributed around the mean of the start values from the training data, with half the standard deviation of the start values in the training data:

$$\text{seed} \sim \mathcal{N}(\mu, 0.5 \cdot \sigma)$$

From this seed, the generator recursively predicts each timestep and takes that timesteps output as the input for the next timestep.

However, in this first basic model, the results weren't great. As shown in Figure 20, the generated sequences tended to **rise** or **fall** too **early**, whereas real sequences often remain constant for longer periods (Figure 17).

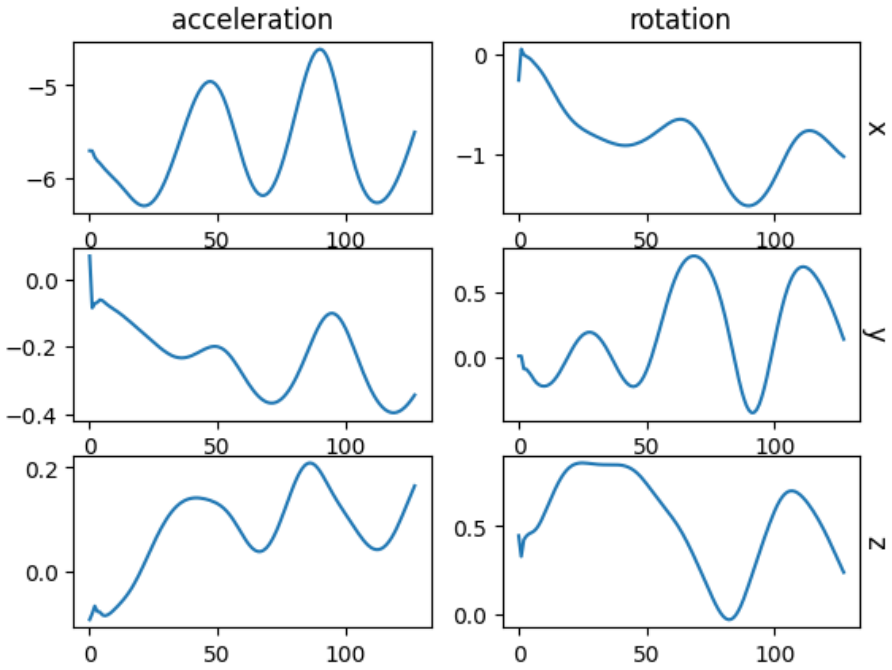


Figure 20: Generated Sequence with simple RNN model

Center Pull Correction

To counter this issue, I implemented a "*pull-to-center*" correction during generation. Each timestep, the generator's output is **pulled back** towards the *average sensor value* (Figure 17), with stronger correction when the standard deviation of the training data is low (i.e., when the training data agrees on the signal) and weaker when it's high:

$$x_t^{\text{corr}} = x_t + \lambda \cdot \frac{\mu - x_t}{\sigma + \epsilon}$$

where, μ and σ are respectively the mean and standard deviation of the real data at that timestep, and λ controls the *strength of the correction.*

This approach was **hard to tune** — if λ was too **strong**, all generated sequences looked the *same*. If too **weak**, I ended up with the *same problem as before.*

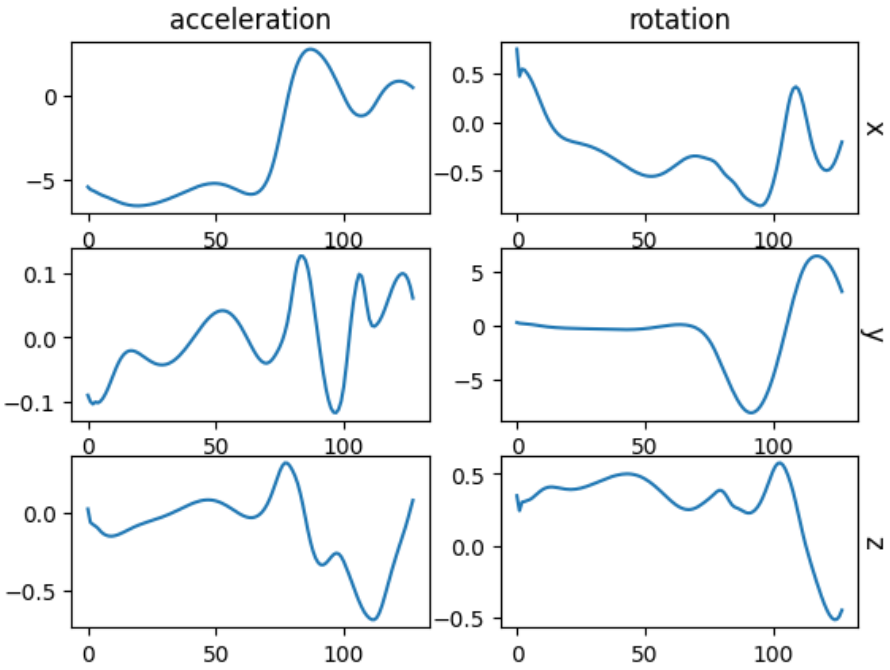


Figure 21: Generated Sequence with pull-to-center generator

Flatness Rewarding Loss

I noticed the generator was constantly trying to change the values even at timesteps where in the original data the sequence is rather flat. Therefore I tried another ap-

proach and created a *custom loss* which **rewarded flatness** and was implemented as:

$$\mathcal{L}_{\text{custom}} = \mathcal{L}_{\text{MSE}} + \lambda \cdot \mathcal{P}_{\text{flat}}$$

$$\mathcal{P}_{\text{flat}} = \frac{1}{T-1} \sum_{t=1}^{T-1} |x_{t+1} - x_t|^2$$

This worked to some extent, making the sequences *flatter* — but mostly toward the end, and not at the beginning where I needed it.

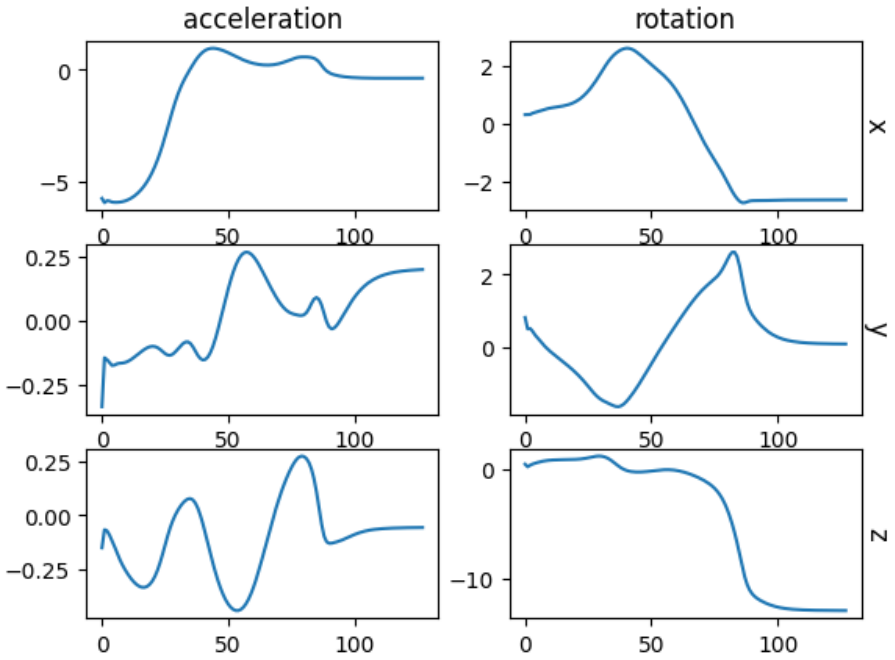


Figure 22: Generated Sequence with flatness-rewarding generator

Teacher Forcing

Up until this point, all changes were targeting the *generator*. It became clear this wasn't enough. So I looked into improving the **training process** itself — and found Brownlee 2017 whose article explains **teacher forcing.**

Until now, during the training, we have always fed the real data for each timestep. **Teacher forcing** changes that so sometimes we feed the *real* data and sometimes

we feed the *generated* data from the previous step. This helps prevent *error accumulation* during training and teaches the model to work with *its own generated data,* which it has to do during the generation of new sequences.

After some experimenting, I found an exponentially decaying schedule for teacher forcing to work best:

$$\textit{real\_data\_ratio}(\text{step}) = \exp\left(-5 \cdot \frac{\text{step}}{\text{total\_steps}}\right)$$

In the beginning only real data gets fed, i.e. a *real-data-ratio* of **1**. This ratio then **exponentially decreases** until in the last time step only generated data gets passed through.

As shown in Figure 23, this led to much more accurate and realistic sequences. And these generated sequences even have a good amount of variation between them.

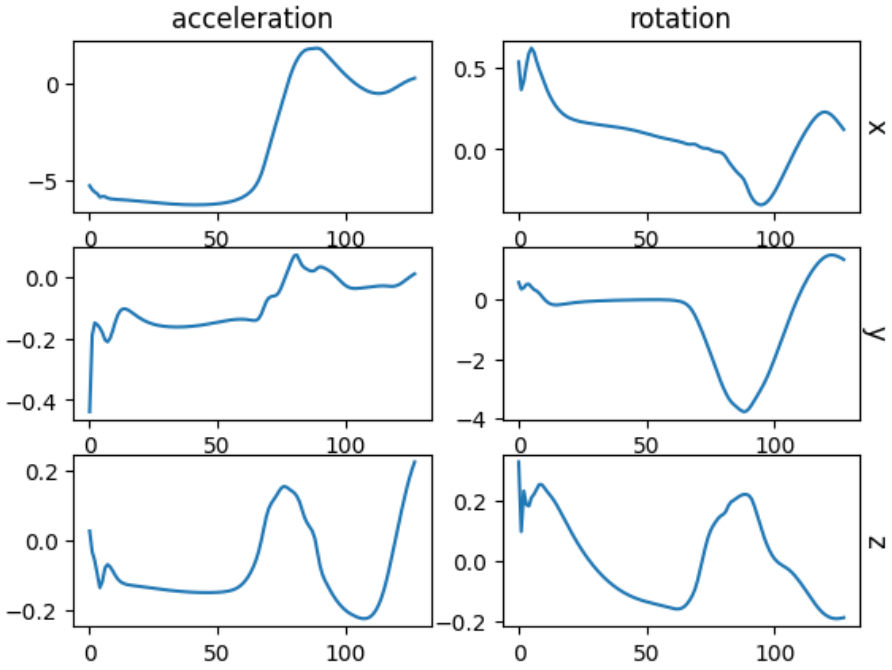


Figure 23: Generated Sequence with teacher forcing

Label Embedding

Before we can use the *generated* data as *training* data for our model from the first task, we have to **label it.**

To allow **label-conditional generation,** I embedded the 12 braking states into a vector and concatenated this embedding with the sensor data as input to the LSTM. This allowed the model to generate sequences specific to **each class**. The performance of the final model — using the metric defined in Section 3.1 — is summarized in the table below:

Label	Avg. Error
0	0.1909
1	0.4630
2	0.3774
3	0.2928
4	0.1809
5	0.2396
6	0.1542
7	0.3271
8	0.1973
9	0.3786
10	0.4329
11	1.9298

Table 6: Average generation error per label

3 Theoretical interpretation

For all three models, the test accuracy using the `test.pkl` data set was around **50%** in the best case. Introducing the synthetic data produced by the GAN or the RNN to the original dataset didn't solve this problem, as it didn't make the performance of the best model any better (cf. *augmentation.ipynb*).

A visual comparison of the training dataset and the testing dataset shows that a possible reason for the model not being strong enough can be explained by the

difference between both datasets. The following graph, which uses the *mean* and the *standard deviation* in the testing data, shows this difference.

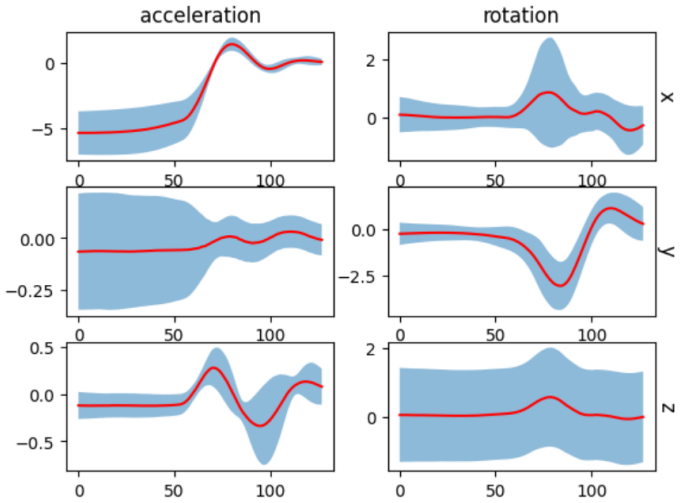


Figure 24: The deviation plot over time for the data in test.pkl. The red graph is the mean μ_t over time t. The blue area is the space between $\mu_t - \sigma_t$ and $\mu_t + \sigma_t$.

Comparing Figure 17 to Figure 24 shows the difference between both which appears more clearly in the **rotation** over the x and the z axis and in the **acceleration** over the y axis. This problem could be due to **different car models, weights** and **brakes** used in both experiment sets.

In order to have a model that is best suited for real life use, it is necessary for the *training* data to include data from *similar cars* in *similar conditions* to the ones being tested. Looking for **bias** in the *training* dataset and trying to **balance the input** would also be helpful for creating a better model.

Conclusion and perspectives

The first part of our work aimed to explore the potential of various known deep learning architectures to classify braking system conditions using labeled vehicle sensor data. A comparative analysis has been done between FCNs, 2D CNNs, and LSTM networks, revealing interesting insights. While FCNs gave excellent results under controlled conditions, their generalization capacity was limited. LSTMs showed great potential, but their susceptibility to class imbalance hindered their usage. The 2D CNN showed balanced results, delivering relative robustness across different data splits.

In the second phase of our project, the focus shifted toward generative AI, having as a purpose augmenting of the training data and addressing the issue of class imbalance. In order to achieve this, two methodologies have been discussed. First, Conditional Generative Adversarial Networks (cGANs) were effective in producing labeled samples especially for dominant classes, though they struggled with rare classes. The second discussed possibility for synthetic data generation was RNN-based sequence generators. Coupled with custom techniques such as teacher forcing and custom flatness penalties, RNNs revealed promising capacity for learning temporal patterns, producing smoother and more realistic time series over time.

To assess the quality of the generated synthetic data, both quantitative and qualitative evaluation methods were used. A primary metric was a least-squares-based distance between each generated sequence and the class-specific mean curve derived from real data. Although simplistic, this measure helped evaluate how closely the artificial data aligned with the general dynamics of each braking condition. Visual inspection of generated time series also helped evaluate its quality by revealing patterns. However, this metric has inherent limitations, especially its inability to take temporal correlations and intra-class variability into account.

Another more implicit metric was the impact of the generated data when added to the training set of the classification models. Unfortunately, despite improvements in diversity, enriching the dataset with artificial samples did not significantly enhance classification performance on the independent test set (as showed in the `augmentation.ipynb` notebook). This result puts emphasis on a subtle but very important distinction : while the synthetic data may appear realistic alone, its distributional alignment with test data remains limited.

To further improve the generative models, many directions could be explored. For example, to address the uneven performance of the GAN across different labels,

one can provide more balanced input distributions or targeted sampling strategies. Additionally, the generator architecture could be enhanced by using larger deconvolution kernels or deeper layers. Post-processing can also be explored, especially through Gaussian smoothing or temporal upsampling followed by downsampling in order to refine the output. Lastly, one could adopt more advanced quality metrics that consider temporal structure, or we could also integrate feedback from classification performance directly into the generation process.

References

Brownlee, Jason (2017). *Teacher Forcing for Recurrent Neural Networks*. https://machinelearningmastery.com/teacher-forcing-for-recurrent-neural-networks/. Accessed: 2025-06-17.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. MIT Press. URL: http://www.deeplearningbook.org/.

Goodfellow, Ian J. et al. (2014). "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems*. Ed. by Z. Ghahramani et al. Vol. 27. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2014/file/f033ed80deb0234979a61f95710dbe25-Paper.pdf.

Heusel, Martin et al. (2017). "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium". In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/8a1d694707eb0fefe65871369074926d-Paper.pdf.

Image References

- Figure 18: A Gentle Introduction to Generative Adversarial Networks (GANs) by Jason Brownlee

Appendix

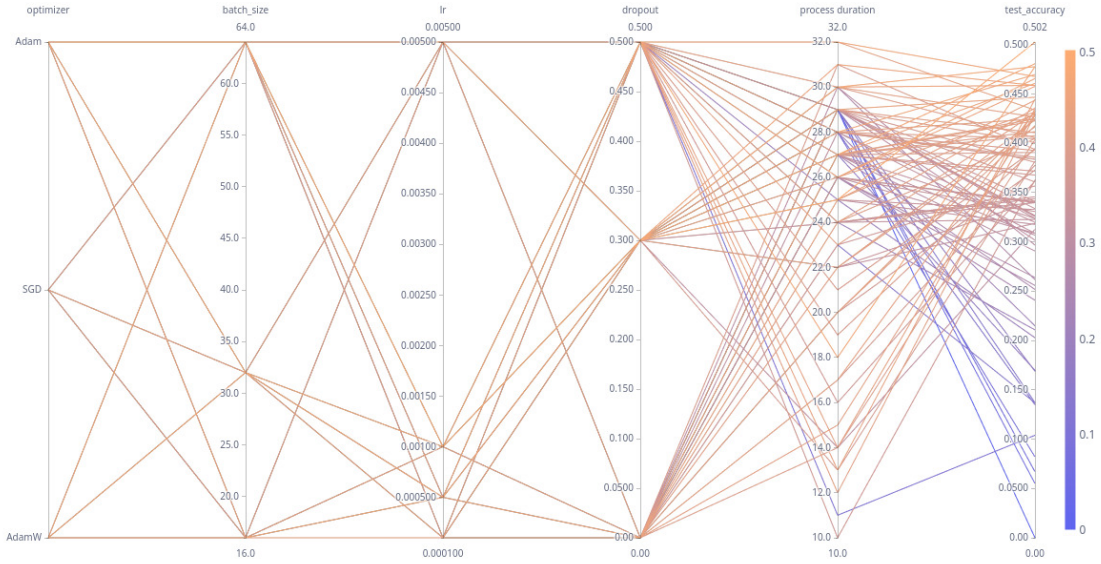
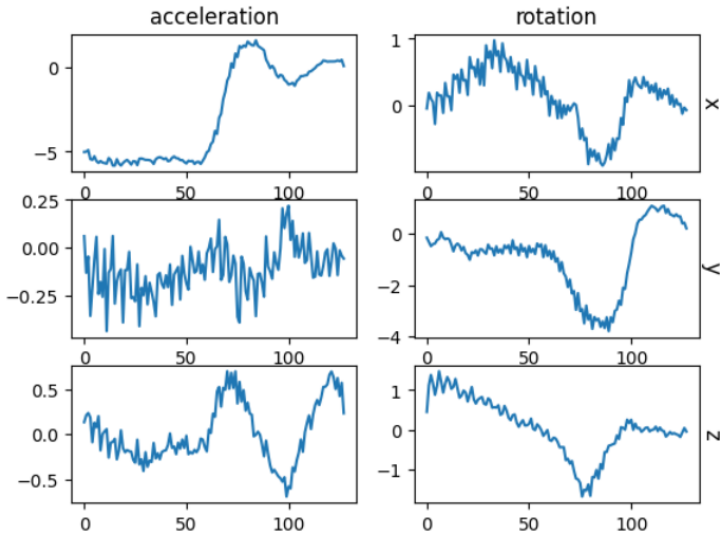
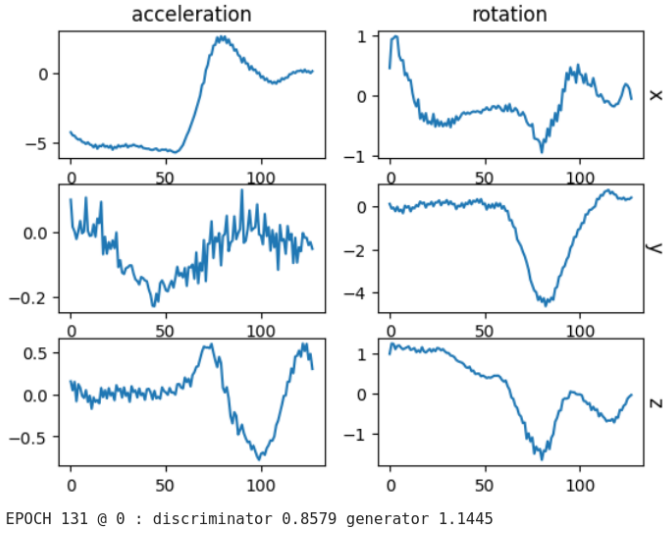


Figure 25: Parallel coordinate plot for the experiment with the 2D CNN

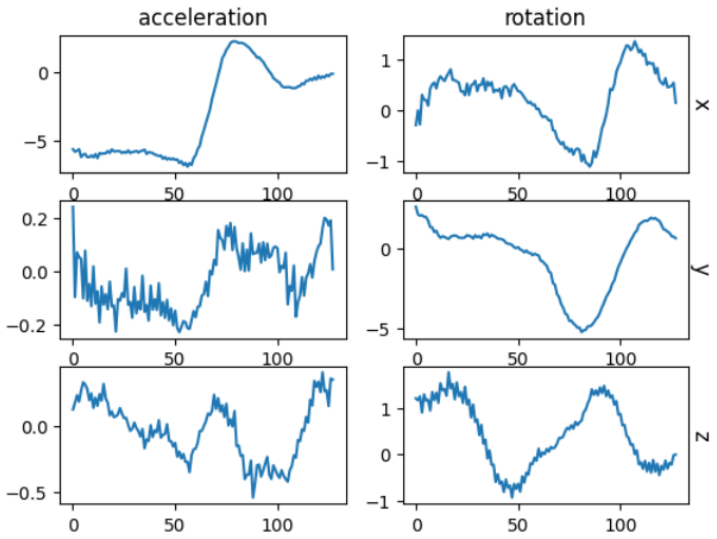
GAN evolution



Synthetic data after 50 epochs

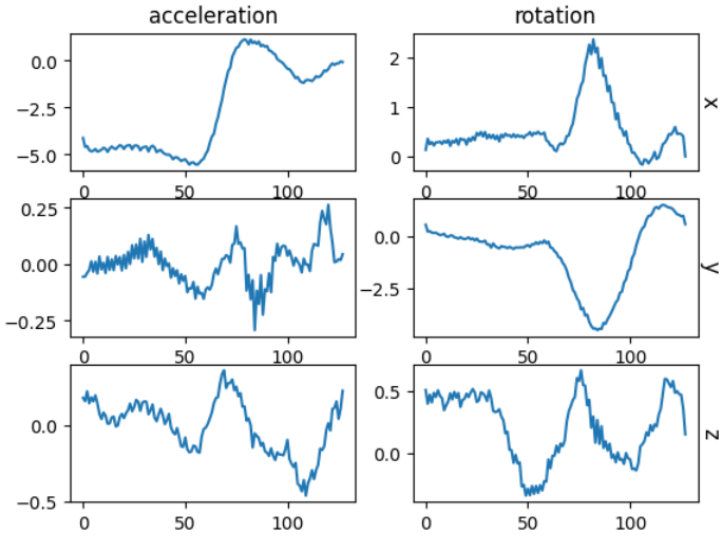


Synthetic data after 130 epochs



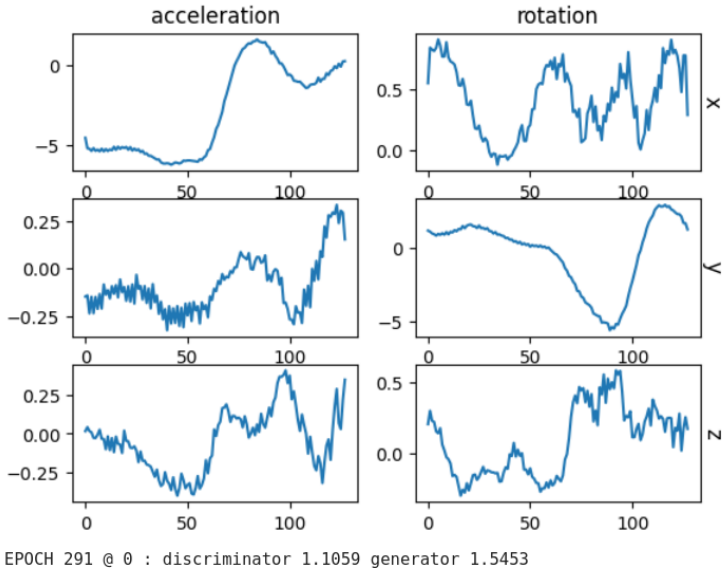
EPOCH 201 @ 0 : discriminator 1.4102 generator 0.9714

Synthetic data after 200 epochs



EPOCH 251 @ 0 : discriminator 1.2464 generator 0.9314

Synthetic data after 250 epochs



Synthetic data after 300 epochs